

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
 Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 – Pattern Recognition and Text Processing

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Pattern Recognition and Text Processing
Weightage:	40% of CIA		
CO5 Statement:	Analyse regular expression patterns. (BL-3)		
Student Name:	T S SAI LAHARI	Reg. No.:	192325137
Section / Batch:	Artificial Intelligence & Machine Learning	Date:	11/07/2026

Code of Conduct

I T.S. Sai Lahari (192325137) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: T.S. Lahari

Instructions

- Show all intermediate steps, calculations, state transitions, and reasoning wherever applicable.
- Use only the Python re (Regular Expressions) module to solve the given problems.
- Clearly mention the Regular Expression (Regex) pattern used before writing the corresponding Python program.
- Display the required output for each question, including extracted values, validation results, counts, and positions wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Generation	Pattern Matching Information Extraction	1
Search for a String	Keyword Search and Text Analysis	2
Split the documentation into sentences and words	Text Tokenization and Sentence Segmentation	3
Email and Strong Password Validation	Data Validation and Format Verification	4
Compile Once, Validate Many	Pattern Reusability and Performance Optimization	5

Annexure A – Pattern Recognition and Text Processing

Section 1: Regular Expression Pattern Generation

Student Details:

Name: John Doe

Email: john.doe123@gmail.com

Mobile: +91-9876543210

Password: P@ssw0rd123

Date of Birth: 15/08/2004

Register Number: 23AIML1056

Department: Artificial Intelligence and Machine Learning

For the given student details formulate Regex patterns for the following:

Email ID

Mobile Number

Strong Password

Date of Birth (DD/MM/YYYY)

Register Number

Regular Expression Pattern Generation

Student Details

- Name: John Doe
- Email: john.doe123@gmail.com
- Mobile: +91-9876543210
- Password: P@ssw0rd123
- Date of Birth: 15/08/2004
- Register Number: 23AIML1056
- Department: Artificial Intelligence and Machine Learning

Regular Expression Patterns

1. Email ID

Regex Pattern:

`^[a-zA-Z0-9.%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$`

Description: Validates an email address containing a username, '@' symbol, domain name, and domain extension.

2. Mobile Number

Regex Pattern:

`^\+91-[6-9]\d{9}$`

Description: Validates an Indian mobile number with country code (+91) followed by a 10-digit mobile number starting with 6, 7, 8, or 9.

3. Strong Password

Regex Pattern:

`^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@#%&+=!]).{8,}$`

Description: Validates a strong password containing at least one uppercase letter, one lowercase letter, one digit, one special character, and a minimum length of 8 characters.

4. Date of Birth (DD/MM/YYYY)

Regex Pattern:

`^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/(19|20)\d{2}$`

Description: Validates a date in the DD/MM/YYYY format.

5. Register Number

Regex Pattern:

`^d{2}[A-Z]{4}\d{4}$`

Description: Validates a register number consisting of 2 digits, followed by 4 uppercase letters, and ending with 4 digits.

Validation of Student Details

Field	Value	Status
Email ID	john.doe123@gmail.com	Valid
Mobile Number	+91-9876543210	Valid
Password	P@ssw0rd123	Valid
Date of Birth	15/08/2004	Valid
Register Number	23A1ML1056	Valid

Section 2: Search for a String

Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.

Write a Python program using the `re` module to perform the following operations for the word "AI":

1. Find the first occurrence of the word "AI" using `re.search()`.
2. Display the starting position of the first occurrence.
3. Display the ending position of the first occurrence.
4. Display the total number of times the word "AI" appears in the passage.
5. Print an appropriate message if the word is not found.

Problem Statement

Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.

CODE AND OUTPUT:

```
1 import re
2 text = "Artificial intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis. In
3 word = "AI"
4 # Find the first occurrence
5 match = re.search(word, text)
6
7 if match:
8     print("First occurrence of 'AI' found.")
9     print("Starting Position:", match.start())
10    print("Ending Position:", match.end())
11
12 # Count total occurrences
13 occurrences = re.findall(word, text)
14 print("Total number of occurrences:", len(occurrences))
15
16 if not occurrences:
17     print("The word 'AI' was not found.")
```

Python 3.10.4 Shell

```
# C:\Users\hp\Documents>python c:\Users\hp\AppData\Local\Programs\Python\Python11\python.exe c:\Users\hp\Documents\SLP\AI.py
First occurrence of "AI" found
Starting Position: 20
Ending Position: 22
Total number of occurrences: 6
# C:\Users\hp\Documents>
```

Python 3.10.4 Shell

10:07:29 AM

Below the previous passage and Write a Python program using the `re.split()` method to perform the following operations:

- Split the passage into sentences using appropriate punctuation (., ?, !) as delimiters.
- Count and display the total number of sentences.
- Split the passage into words by considering one or more whitespace characters as delimiters.
- Count and display the total number of words.
- Display the list of extracted sentences and words.

Problem Statement

Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.

CODE AND OUTPUT:

```

1 # Python program to split a string into sentences and words using regular expressions
2
3 import re
4
5 # Sample text
6 text = "Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand."
7
8 # Splitting text into sentences using punctuation as delimiters
9 sentences = re.split(r'(?=[.?!])', text)
10
11 # Counting total number of sentences
12 total_sentences = len(sentences)
13
14 # Splitting text into words using whitespace as delimiters
15 words = re.split(r'\s+', text)
16
17 # Counting total number of words
18 total_words = len(words)
19
20 # Displaying results
21 print("Total number of sentences:", total_sentences)
22 print("Total number of words:", total_words)
23
24 # Displaying the list of extracted sentences and words
25 print("Sentences:", sentences)
26 print("Words:", words)

```

System 1: Email and Strong Password Validation

A website requires users to register by providing an Email ID and a Strong Password. Write a Python program using the `re` module to validate the following inputs.

Validation Criteria:

Email ID

- Must begin with one or more letters or digits.
- May contain `_`, `%`, `+`, or `=` before the `@` symbol.
- Must contain a valid domain name.
- Must end with a valid domain extension such as `.com`, `.org`, `.edu`, or `.in`.

Strong Password

- Must contain at least 8 characters.
- Must contain at least one uppercase letter.
- Must contain at least one lowercase letter.
- Must contain at least one digit.
- Must contain at least one special character from `@`, `#`, `%`, `&`, `!`, or `*`.
- Must not contain any whitespace characters.

Problem Statement

A website requires users to register by providing an Email ID and a Strong Password. Write a Python program using the re module to validate the following inputs.

Validation Criteria

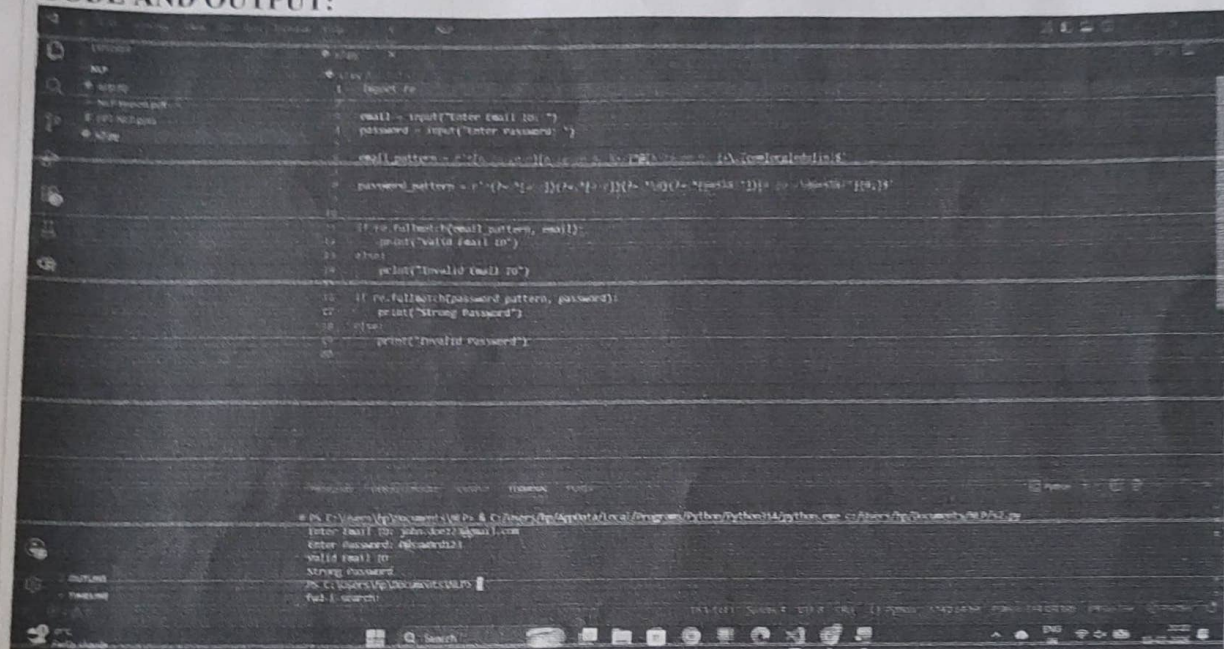
Email ID

- Must begin with one or more letters or digits.
- May contain ., _, %, +, or - before the @ symbol.
- Must contain a valid domain name.
- Must end with a valid domain extension such as .com, .org, .edu, or .in.

Strong Password

- Must contain at least 8 characters.
- Must contain at least one uppercase letter.
- Must contain at least one lowercase letter.
- Must contain at least one digit.
- Must contain at least one special character from @, #, \$, %, &, !, or *.
- Must not contain any whitespace characters.

CODE AND OUTPUT:



```
1 import re
2
3 email = input("Enter Email ID: ")
4 password = input("Enter Password: ")
5
6 email_pattern = r'^[a-zA-Z0-9_+-%]{1,64}@([a-zA-Z0-9-]{1,63}\.([a-zA-Z]{2,6})$|([a-zA-Z0-9-]{1,63})$)'
7 password_pattern = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[a-zA-Z\d@$!%*?&]{8,}$'
8
9 if re.fullmatch(email_pattern, email):
10     print("Valid Email ID")
11 else:
12     print("Invalid Email ID")
13
14 if re.fullmatch(password_pattern, password):
15     print("Strong Password")
16 else:
17     print("Invalid Password")
18
19 # Python 3.9.6 Shell
20 Enter Email ID: john.doe@gmail.com
21 Enter Password: @hacker123!
22 Valid Email ID
23 Strong Password
24 PS C:\Users\johndoe> python3.9\python.exe c:\Users\johndoe\Documents\Python\3.9\3.9.6\python.exe
```

Section 5: Compile Once, Validate Many

A company receives 1,000 email IDs from its customers and needs to validate each one efficiently. Instead of compiling the same regular expression for every email, write a Python program using the re.compile() method to compile the pattern once and reuse it to validate all the email IDs.

Problem Statement

A company receives 1,000 email IDs from its customers and needs to validate each one efficiently. Instead of compiling the same regular expression for every email, write a Python program using the re.compile() method to compile the pattern once and reuse it to validate all the email IDs.

CODE AND OUTPUT:

```
1 # Importing the re module and creating the pattern
2 import re
3 # Creating the email validation pattern
4 email_pattern = re.compile(r'^[a-zA-Z0-9]+@[a-zA-Z0-9]+\.[a-zA-Z]{2,}$')
5
6 # A sample list of email addresses
7 emails = [
8     "john.doe@gmail.com",
9     "student@college.edu",
10     "info@company.com.au",
11     "user@domain.org",
12     "invalidmail",
13     "email.com",
14     "user@.com"
15 ]
16
17 # Print the email validation results
18 print("Email Validation Results:")
19
20 # Iterate over the emails
21 for email in emails:
22     if email_pattern.fullmatch(email):
23         print(email, "-> Valid Email ID")
24     else:
25         print(email, "-> Invalid Email ID")
26
27 # Running the script
28
29 PS C:\Users\p\Documents\MLP>
30 PS C:\Users\p\Documents\MLP> & C:\Users\p\AppData\Local\Programs\Python\Python311\python.exe c:\Users\p\Documents\MLP\4.py
31 Email Validation Results:
32 john.doe@gmail.com -> Valid Email ID
33 student@college.edu -> Valid Email ID
34 info@company.com.au -> Valid Email ID
35 user@domain.org -> Valid Email ID
36 invalidmail -> Invalid Email ID
37 email.com -> Invalid Email ID
38 user@.com -> Invalid Email ID
39 PS C:\Users\p\Documents\MLP>
```


Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q. No. / Criteria	Conceptual Understanding	Accuracy of Answer	Application of Knowledge	Completeness of Response	Clarity & Presentation	Sub. Total	Total %
1						19	94.
2						19	
3	4	4	4	3	4	19	
4						19	
5					3	18	

Faculty In-charge	Course Coordinator	Program Director
Signature: <i>[Signature]</i>	Signature: _____	Signature: _____
Date: <i>28/07</i>	Date: _____	Date: _____